AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING
i-CREDIT HOURS ENGINEERING PROGRAMS

University Management System: Phase 1 Report

CSE342: Agile Software Engineering

Submitted to:
- Dr. Mohamed Hassan Mahmoud El Gazzar
- TA. Hazem Zaki
- TA. Abdelrahman Salah

Team Members

1- Omar Ahmed Abouraia 23P0100
2- Hossam Osama Hussieny 23P0010
3- Mariam Ibrahim Assr 23P0123
4- Mariam Tarek Farouk 23P0214
5- Lujain Mohamed 23P0187

Table of Contents

1. Project Introduction 3
2. Team Organization & Workspace 3
3. General Definition of Done (DoD) 3
4. High-Level Requirements: 4
   1. Facilities Module: Detailed Planning 4
      1.1 Feature Prioritization (MoSCoW) 4
      1.2 User Story Splitting Technique 5
      1.3 Change Anticipation 5
      1.4 Teamwork & Integration 6
      1.5 Sprint 1 Backlog (MVP Items) 6
   2. Community Module: Detailed Planning 7
      2.1 Feature Prioritization (MoSCoW) 7
      2.2 User Story Splitting Technique 8
      2.3 Change Anticipation 8
      2.4 Teamwork & Integration 9
      2.5 Sprint 1 Backlog (MVP Items) 9
   3. Staff Module: Detailed Planning 10
      3.1 Feature Prioritization (MoSCoW) 10
      3.2 User Story Splitting Technique 11
      3.3 Change Anticipation 11
      3.4 Teamwork & Integration 11
      3.5 Sprint 1 Backlog (MVP Items) 12
   4. Curriculum Module: Detailed Planning 13
      4.1 Feature Prioritization (MoSCoW) 13
      4.2 User Story Splitting Technique 14
      4.3 Change Anticipation 14
      4.4 Teamwork & Integration 15
      4.5 Sprint 1 Backlog (MVP Items) 15
5. Appendix: Project Resources 15

1. Project Introduction

This report outlines the Phase 1 planning for the University Management System, developed using the **Scrum Framework**. The system aims to modernize university operations by providing a centralized platform for students, faculty, and administrators. Our agile approach focuses on delivering a Minimum Viable Product (MVP) that addresses core academic and administrative needs while remaining flexible for future enhancements.

2. Team Organization & Workspace

The project is managed by a team of 5 members, utilizing **Jira Software** as our primary Agile project management tool. To ensure seamless collaboration:

- **Unified Workspace:** All modules are integrated into a single Jira Scrum project.
- **Component-Based Architecture:** We utilized Jira **Components** to organize the backlog into four distinct areas: Facilities, Curriculum, Staff, and Community.
- **Collaboration:** Weekly syncs are held to manage cross-module dependencies, such as linking Staff profiles to Curriculum course assignments.

3. General Definition of Done (DoD)

The team has agreed upon the following **DoD** to ensure quality across all modules (Curriculum, Facilities, Staff, Community) before an item is considered "Done":

1. **Format:** The story follows the "As a... I want... So that..." structure.
2. **Acceptance Criteria:** At least 3 testable ACs are defined in the Jira ticket.
3. **Attributes:** The story is assigned to a **Component** and linked to a **Parent Epic**.
4. **Estimation:** The story has a **Priority** level (High/Medium/Low) and a Story Point estimate.
5. **Peer Review:** The story has been reviewed and approved by another team member to ensure it doesn't conflict with other modules.
6. **Change Anticipation:** The story description includes a note on how the feature can scale or adapt in future sprints.

4. High-Level Requirements:

1. Facilities Module: Detailed Planning

1.1 Feature Prioritization (MoSCoW)

We used the **MoSCoW technique** to prioritize the Facilities Module features. This ensures that the system provides essential infrastructure management functionalities in the MVP.

Category	Feature (User Story)	Priority	Reason
Must Have (MVP)	View Room Availability/Schedule	High	Core infrastructure feature required to begin any academic activity.
Must Have (MVP)	Manage Student Records	High	Essential for digitizing paper-based administrative processes.
Must Have (MVP)	Allocate Equipment/Resources	High	Required for tracking university assets across departments.
Should Have	Report Maintenance Issues	Medium	Critical for facility upkeep but can be managed manually in the first week.
Should Have	Handle Admission Applications	Medium	High priority for the Registrar, but not required for active student registration.
Could Have	Booking Notifications	Low	Enhances user experience but the schedule view is the primary source of truth.
Won't Have (Phase 1)	AI Resource Optimization	Low	Advanced automation planned for later development cycles.

1.2 User Story Splitting Technique

To maintain agility and follow the **INVEST** criteria, the Facilities Module utilized **Functional Decomposition** and **User Role Splitting**.

Technique: Functional Decomposition

- **Large User Story (Epic):** "As a user, I want to manage all university physical and digital infrastructure."
- **Split A (View Path):** "As a Student, I want to view available classrooms for study sessions."
- **Split B (Action Path):** "As a Professor, I want to reserve a laboratory for a specific class time."
- **Split C (Admin Path):** "As an Admin, I want to manage the master classroom schedule."

Reasoning: Splitting by function allowed us to prioritize core operations—viewing and scheduling—as these are essential prerequisites before secondary features like maintenance reporting or advanced filtering are developed.

1.3 Change Anticipation

The Facilities Module is designed with flexible architecture to handle institutional growth:

- **Dynamic Scheduling:** The scheduling engine is built to handle real-time modifications. This ensures that if university timetables or room capacities change, the system only requires a configuration update rather than a core code rewrite.
- **Scalable Resource Schema:** The resource allocation system is built with a "Generic Asset" model, allowing the university to add new types of resources (e.g., software licenses, lab equipment, or vehicles) without major database redesign.

1.4 Teamwork & Integration

The Facilities Module acts as the physical foundation for other academic modules:

- **Curriculum Integration:** Classroom schedules are directly linked to the **Curriculum Module** to ensure that lectures and exams are automatically assigned to rooms based on capacity and course size.

- **Staff Integration:** We collaborated with the **Staff Module** lead to ensure that room booking requests are automatically validated against the staff directory to verify authorization levels.

1.5 Sprint 1 Backlog (MVP Items)

The following User Stories have been moved to the Sprint Backlog to achieve the MVP:

1. **View Available Classrooms & Schedules:** Provides visibility into the university's physical infrastructure.

2. **View Room Schedule:** Allows students and staff to see the daily and weekly timeline for specific halls.

3. **Manage Classroom Schedule:** Allows admins to finalize the master room plan.

4. **Manage Student Records:** Establishes the core administrative database for the system.

5. **Allocate Equipment:** Enables the initial tracking of software and hardware resources.

2. Community Module: Detailed Planning

2.1 Feature Prioritization (MoSCoW)

The Community Module was prioritized to establish the essential communication loop between students and faculty. We applied the **MoSCoW technique** as follows:

Category	Feature (User Story)	Priority	Reason
Must Have (MVP)	Send Message to Professor	High	Core communication feature allowing students to seek guidance.
Must Have (MVP)	Reply to Student Messages	High	Enables the necessary two-way interaction for academic support.
Must Have (MVP)	View Message Inbox	High	Essential for users to read and manage their communications.
Should Have	Post/View Announcements	Medium	Important for university-wide updates but secondary to direct chat.
Could Have	Schedule Meetings	Low	Enhances organization but can be managed via direct messaging initially.
Could Have	Parent Portal (Progress/Chat)	Low	Vital for external engagement but planned for a future release phase.
Won't Have	View University Events	Low	An informational feature that does not impact core academic workflows.

2.2 User Story Splitting Technique

The Community Module was split using **User Role Splitting** and **Functional Path Splitting** to separate internal academic communication from external parent engagement.

Technique: Functional Path Splitting

- **Large User Story:** "As a University Member, I want to communicate with all other stakeholders."
- **Split A (Student Path):** "As a Student, I want to message my professor for academic guidance."
- **Split B (Staff Path):** "As a Professor, I want to reply to student inquiries and post announcements."
- **Split C (External Path):** "As a Parent, I want to message teachers and view my child's progress."

Reasoning: By splitting the "Communication" mega-story into functional paths, we isolated the **Student-to-Staff Path** as the MVP. This allows the team to build the core messaging engine before adding the more complex "Parent Portal" security and progress-tracking integrations.

2.3 Change Anticipation

The Community Module is designed with a "Module-Agnostic" communication architecture to handle future needs:

- **Dynamic Routing:** The messaging system is built to pull contact lists directly from the **Staff Module**. This ensures that if the university adds new staff roles (e.g., Career Counselors), the community module automatically includes them in the messaging directory without needing a manual update.
- **Notification Scalability:** The "Announcements" feature is designed to support different "Channels" (e.g., Departmental vs. Campus-wide). This anticipates a future where the university might grow to multiple campuses or specialized institutes.

2.4 Teamwork & Integration

The Community Module relies on deep integration with the Curriculum and Staff data:

- **Curriculum Integration:** The "Parent View Progress" story (Planned for Sprint 2) is mapped to pull real-time data from the **Curriculum Module's** gradebook.
- **Staff Integration:** We coordinated with the **Staff Module** lead to ensure that the "Message Professor" button is embedded directly onto the staff profile pages, creating a seamless user experience.

2.5 Sprint 1 Backlog (MVP Items)

The following User Stories have been moved to the Sprint Backlog to establish the MVP:

1. **Send Message to Professor:** Initiates the student-teacher communication channel.
2. **Reply to Student Messages:** Completes the interaction loop for faculty.
3. **View Message Inbox:** Provides a centralized view for all received communications.

3. Staff Module: Detailed Planning

3.1 Feature Prioritization (MoSCoW)

The Staff Module was prioritized to ensure that the foundational directory and teaching assignments are ready for the academic cycle. We applied the **MoSCoW technique** as follows:

Category	**Feature (User Story)**	**Priority**	**Reason**
Must Have (MVP)	View Staff Directory	High	Essential for students to find and contact their professors.
Must Have (MVP)	View Assigned Courses	High	Critical for identifying instructors responsible for specific subjects.
Must Have (MVP)	Manage TA Responsibilities	High	Required for defining TAs' roles in grading, labs, and office hours.
Should Have	Maintain Staff Records	High	Vital for HR to ensure directory data remains accurate and updated.
Could Have	View Performance Records	Medium	Useful for faculty evaluation but not critical for the initial launch.
Could Have	Payroll & Leave Requests	Medium	Important HR functions that can be phased in after the directory.
Won't Have	Research Publications	Low	Supports academic tracking but is secondary to core operations.

3.2 User Story Splitting Technique

The Staff Module was split using **User Role Splitting** and **Functional Area Splitting** to break down a large administrative scope into manageable Epics and Stories.

Technique: User Role Splitting

- **Large User Story:** "Manage all university staff data and services."
- **Split A (The Directory Path):** "As a Student, I want to view my professor's office hours and contact info."
- **Split B (The HR Path):** "As an Admin, I want to maintain staff records and payroll."
- **Split C (The TA Path):** "As a TA, I want to manage my assigned lab and grading duties."

Reasoning: By splitting based on the user (Student vs. HR vs. TA), we were able to prioritize the **Directory Path** for the MVP, as it provides the most immediate value to the student body.

3.3 Change Anticipation

The Staff Module is designed to handle future organizational growth without system redesign:

- **Role Scalability:** The "TA Roles and Responsibilities" story is built with a flexible role-assignment logic. If the university introduces new staff categories (e.g., "Guest Lecturers" or "Research Assistants"), the system can accommodate these new roles via simple attribute changes rather than a database overhaul.
- **Departmental Flexibility:** The directory is structured by "Component" (Department), allowing the university to add new faculties or administrative units seamlessly.

3.4 Teamwork & Integration

The Staff Module serves as a data provider for several other modules:

- **Curriculum Integration:** We collaborated closely with the **Curriculum Module** lead to ensure that when a Professor is assigned to a course, their profile (Office Hours, Bio) is automatically pulled into the course view.
- **Community Link:** The staff directory links directly to the **Community Module** messaging system, allowing students to initiate a chat directly from a staff profile.

3.5 Sprint 1 Backlog (MVP Items)

The following User Stories have been moved to the Sprint Backlog to establish the MVP:

1. **View Staff Directory:** Provides a centralized, searchable list of all faculty members.

2. **View Assigned Courses per Staff Member:** Enables transparency regarding teaching responsibilities.

3. **Manage TA Roles and Responsibilities:** Defines the specific duties for teaching assistants.

4. **Maintain Staff Records:** Allows administrators to ensure information remains current and accurate.

4. Curriculum Module: Detailed Planning

4.1 Feature Prioritization (MoSCoW)

We used the **MoSCoW technique** to prioritize the Curriculum features. This ensures the system provides a working academic cycle immediately.

Category	Feature (User Story)	Priority	Reason
Must Have (MVP)	Semester Course Offering	High	Essential for Unit Heads to activate the curriculum for the term.
Must Have (MVP)	Rule-Based Registration	High	Core engine required for students to enroll while enforcing GPA limits.
Must Have (MVP)	Grade Entry & GPA Calc	High	Critical for instructors to record marks and determine academic standing.
Should Have	Add/Drop & Withdrawal	Medium	Necessary for schedule flexibility but can be enabled after initial enrollment.
Should Have	Exam Schedule View	Medium	Important for student planning but secondary to the registration phase.
Could Have	Anonymized Course Surveys	Low	Provides quality feedback but is not a blocker for core system operations.
Could Have	Prerequisite Mapping	Low	Enhances validation logic for future complex academic paths.
Won't Have (Phase 1)	Advanced SIS Automation	Low	Complex third-party integrations planned for a future development phase.

4.2 User Story Splitting Technique

To keep our stories manageable and "Small" (following INVEST criteria), we used the **Path-Based Splitting** and **User Role Splitting** techniques.

Example: Splitting "Course Management"

- **Large User Story:** "As a University Member, I want to manage all course-related data."
- **Split A (By Role):** "As a Unit Head, I want to offer courses and assign staff" (Focuses on Administrative setup).
- **Split B (By Role):** "As an Instructor, I want to manage materials and grades" (Focuses on Classroom delivery).
- **Split C (By Path):** "As a Student, I want to view my grades" (Focuses on Data consumption).

Reasoning: Splitting by role allowed us to prioritize the Unit Head's setup tasks first, as they are a prerequisite for all other users.

4.3 Change Anticipation

A core strength of the Curriculum module is its design for **Change Anticipation**.

- **Policy Flexibility:** The **Rule-Based Registration** system is built using configurable GPA tiers (e.g., limits for GPA < 2.0 vs. GPA > 3.0). This was designed specifically so that if university policies change next year, the system requires only a **configuration update**, not a core code rewrite.
- **Scalable Assessments:** The gradebook allows dynamic weighting (e.g., changing midterm weight from 20% to 30%), anticipating that different departments have different academic standards.

4.4 Teamwork & Integration

This module demonstrates high cross-team coordination:

- **Staff Integration:** The "Instructor Assignment" story is directly linked to the **Staff Module**. We coordinated with the Staff Module lead to ensure that when a Unit Head assigns a course, the system pulls data from the centralized Staff Profiles.

- **Facilities Link:** We have mapped the "Exam Schedule" and "Transcripts" features to pull room data and administrative signatures from the **Facilities Module**.

4.5 Sprint 1 Backlog (MVP Items)

The following stories have been moved to the current Sprint to achieve the **MVP**:

1. **Semester Course Offering (Unit Head):** To establish the semester's available courses.
2. **Rule-Based Course Registration (Student):** To allow students to join courses based on GPA eligibility.
3. **Instructor Enrollment Finalization:** To provide the necessary academic approval workflow.
4. **Grade Entry & GPA Calculation:** To facilitate the recording of marks and academic standings.
5. **Student Progress Tracking:** To allow students to see their results and materials.

---

5. Appendix: Project Resources

As per the evaluation criteria, all project planning artifacts and source control links are provided below:

- **GitHub Repository:** [**GitHub URL**]
    - *Note: This repository contains the Phase 1 documentation, README, and will host the implementation code in future phases.*
- **Jira Scrum Workspace:** [**Jira URL**]
    - *Note: The Jira space includes the full Product Backlog, Epic structures, and the active MVP Sprint (Sprint 1).*